Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Üçgen türü problemi (önceki dersin ödevi)
  - Kenar uzunlukları a, b, c olarak okunur. Kenarların geçerli bir üçgen oluşturduğu varsayılır.
  - Karşılaştırma operatörleri her zaman iki operand alır. Bir karar kutusunda `a = b = c` ya da `a < b < c` gibi üçlü karşılaştırma yapılamaz.
  - Çözüm iç içe kararlarla kurulur:
    - `a = b` doğruysa: `b = c` doğruysa **eşkenar**, değilse **ikizkenar**.
    - `a = b` yanlışsa: `b = c` doğruysa **ikizkenar**, değilse `a = c` kontrol edilir. Doğruysa **ikizkenar**, değilse **çeşitkenar**.
  - Tüm eşitlik kontrolleri yanlış çıktığında "hiçbiri eşit değil mi" diye tekrar sormaya gerek yoktur. Geriye tek seçenek kalmıştır, sonuç doğrudan çeşitkenardır.
  - Toplam durumlar: tek bir eşkenar durumu, üç ikizkenar durumu (a=b, b=c, a=c) ve tek bir çeşitkenar durumu.
  - Analiz için örnek girdiler: 3, 5, 7 (çeşitkenar), 2, 2, 2 (eşkenar), 3, 3, 4 ve 4, 4, 2 (ikizkenar).
- Dik üçgen kontrolü
  - Pisagor teoremi: bir kenarın karesi diğer iki kenarın kareleri toplamına eşitse üçgen diktir. Hangi kenarın hipotenüs olduğu bilinmediği için üç durum da kontrol edilmelidir: `a*a = b*b + c*c`, `b*b = a*a + c*c`, `c*c = a*a + b*b`.
  - Eşkenar bir üçgen dik üçgen olamaz. Bu yüzden önce üçgenin türünü belirleyip dik üçgen kontrolünü sonra yapmak, eşkenar durumda gereksiz kontrolleri önler.
  - Dik üçgen kontrolünü her ikizkenar ve çeşitkenar kolunun arkasına koymak akış diyagramını uzatır, ama toplam yapılan işlem sayısını artırmaz. Örneğin `a = b` olduğu biliniyorsa yalnızca `c*c = a*a + b*b` kontrolü yeterlidir.
  - Daha uzun kod, bazen daha az işlem yapan ve daha verimli bir algoritma anlamına gelebilir.
- Döngüler (loops)
  - Döngüler, aynı işlemi defalarca yapmak için kullanılır.
  - İki temel döngü türü vardır: for döngüsü ve while döngüsü. While türünün varyasyonları do…while, while ve repeat…until'dir. Bunlar dilden dile değişir.
  - **Adım sayısı belli olan işlerde for, belli olmayan işlerde while kullanılır.**
  - "Adım sayısı belli" demek, işin girdiye bağlı olarak tam olarak kaç kez yapılacağının önceden bilinmesidir (örneğin N kez). Girdiye bağlı olarak kaçıncı adımda biteceği bilinmiyorsa while kullanılır.
- For döngüsü notasyonu (ekrandan, 00:45:45)
  - Ekrana göre kutunun üst kısmında indisin başlangıç değeri (`I ← 1`) ve artış miktarı (`1`), alt kısmında son değer (`100`) yazar.
  - Numaralı kısımlar:
    1. Döngüye giriş.
    2. Döngünün her turunun başladığı nokta.
    3. Turun bitip döngü başına dönüldüğü nokta.
    4. Koşul sağlanmadığında döngüden çıkış.
    5. Her turda yapılan işlemlerin kutusu.
    6. END kutusu: her turun bittiğini sembolik olarak gösterir. Kullanılması zorunlu değildir.

7. Artış/azalış miktarı.
8. İndisin ilk değeri.
9. İndisin alacağı son değer.

- `I ← 1` ile 100 arasında birer artarak dönen bir döngü, 5 numaralı kutudaki işleri 100 kez yapar.
- Başlangıç, bitiş ve artış değerleri sabit olmak zorunda değildir. Değişken de yazılabilir (örn. `N`).
- Artış kullanılıyorsa başlangıç küçük, bitiş büyük olmalıdır. Azalış kullanılıyorsa başlangıç büyük, bitiş küçük olmalıdır.
- Atama oku her zaman değişkene doğrudur (`I ← 1`). Sabite doğru ok çizmek (`I → 1`) hatadır.

- Örnek: N faktöriyel (for döngüsü)
  - Kağıt üzerinde: `n! = n × (n-1) × (n-2) × ... × 1`. Örnek: 4! = 4 × 3 × 2 × 1.
  - Adım sayısı girdiye bağlıdır ama bellidir: iş her zaman N kez yapılır. Bu yüzden for kullanılır.
  - Akış: START → giriş: `N` → `sonuc = 1` → for `I ← N`, adım `-1`, son değer `1` → `sonuc = I * sonuc` → çıkış: `sonuc` → STOP.
  - `sonuc` değişkenine döngüden önce 1 değeri atanmalıdır. Atanmazsa algoritma doğru çalışmaz.
  - Döngü içinde `I` yerine `N` yazmak hatadır: her turda aynı sayıyla çarpılır ve sonuç N^N olur, faktöriyel olmaz.
  - Analiz (N = 5): I = 5 → sonuç 5. I = 4 → 20. I = 3 → 60. I = 2 → 120. I = 1 → 120. I = 0 olunca koşul sağlanmaz ve çıkılır. Sonuç 120'dir.
  - Özel durumlar:
    - N = 0: koşul (0 ≥ 1) sağlanmaz, döngüye girilmez, sonuç 1 olur. Bu doğrudur (0! = 1).
    - N = -1: sonuç yine 1 çıkar, bu istenen bir sonuç değildir. Gerekirse başta negatif sayı kontrolü eklenebilir. Sınavda aksi belirtilmedikçe doğru girdi varsayılır.
  - Son değer 1 yerine 2 yapılabilir. Bu, 1 ile gereksiz çarpmayı önler ve 0 ile 1 için de doğru sonuç verir.
  - Artan sürüm de doğrudur (ekrandan, 01:21:50): `sonuc = 1`, for `I ← 2` (ya da 1), adım `1`, son değer `N`, işlem `sonuc = I * sonuc`.
- While döngüsü notasyonu (ekrandan, 01:21:50)
  - Koşul altıgen bir kutuya yazılır (ekrandan, örn. `X > Y`). Koşul sağlandığı sürece döngü devam eder, sağlanmadığında biter.
  - Numaralı kısımlar: 1 döngüye giriş, 2 döngüden çıkış, 3 her tura başlangıç, 4 döngü içinde yapılacak işlerin kutusu, 5 turun tamamlanıp koşula dönülmesi, 6 koşul.
  - Döngü içine yalnızca işlem kutusu değil, if ya da başka bir döngü de konabilir.
  - Örnek (ekrandan, döngü gövdesi `X = X - 1, Y = Y + 1`):
    - X = 5, Y = 7: 5 > 7 sağlanmaz, döngüye hiç girilmez.
    - X = 7, Y = 5: bir tur döner (X = 6, Y = 6). 6 > 6 sağlanmaz ve döngüden çıkılır.
- Örnek: Bir sayının kaç basamaklı olduğunu bulma (while döngüsü) (ekrandan, 01:31:10)
  - Sayı sürekli 10'a bölünür (`DIV 10`). Kaç kez bölündüğü sayılır.
  - Adım sayısı girilen sayıya göre değişir ve önceden bilinemez: 1453 → 4, 10374 → 5, 1 → 1, 0 → 1. Bu yüzden while kullanılır.
  - Akış: START → `'N sayısını giriniz'` → giriş: `N` → `basamak = 1` → while `N >= 10`: `N = N DIV 10`, `basamak = basamak + 1` → çıkış: `basamak` → STOP.
  - Koşulun doğru sınırı önemlidir:
    - `N < 1` yanlıştır: koşulun yönü ters olduğundan 1453 gibi bir sayıda döngüye hiç girilmez.
    - `N > 10` yanlıştır: N = 10 için sonuç 1 çıkar, oysa doğru sonuç 2'dir.
    - `N >= 10` ya da `N > 9` doğrudur.
  - Analiz: N = 12 → bir tur → N = 1, basamak = 2. N = 5 → döngüye girilmez, basamak = 1.
  - `basamak = 0` ve `N >= 1` alternatifi 0 girildiğinde 0 basamak verir. Bu yanlıştır. Derste bu biçim `N > 1` ile karıştı: `basamak = 0` ve `N > 1` ile 1 için 0, 12 için 1 çıkar.
  - Negatif sayılar: koşul `N ≠ 0` yapılıp basamak 1'den başlatılırsa -12 için 3 bulunur, bu yanlıştır. Basamak 0'dan başlatılırsa negatif sayılar doğru çalışır, ama 0 için ayrı kontrol gerekir. Alternatif olarak sayı negatifse önce -1 ile çarpılıp pozitife çevrilebilir.

Hocanın Özellikle Vurguladığı Kısımlar

- Karşılaştırmalar ikilidir
  - Bir operatör iki operand alır. Üç değeri tek bir koşulda karşılaştırmak mümkün değildir.
- If kollarının birleştirilmesi
  - Her if'in true ve false kolları önce kendi aralarında birleşir, sonra dıştaki if'in koluyla birleşir. İki farklı if'in kollarını tek bir çıkış kutusunda birleştirmek ya da birden fazla STOP kullanmak yanlıştır.
  - Akış diyagramının "güzel görünmesi" önemli değildir. Önemli olan notasyon kurallarına uymaktır.
- For mu, while mı?
  - Adım sayısı belli olan işlerde for, belli olmayanlarda while kullanılır. Adım sayısının girdiye bağlı olması onu belirsiz yapmaz. Faktöriyelde iş her zaman N kez yapılır.
- Önce doğru, sonra verimli
  - Önce doğru sonuç veren algoritma tasarlanır. Sonra en kötü durumda kaç kontrol ve işlem yapıldığına bakılır ve bu sayı azaltılmaya çalışılır.
- Sınır değerlerine dikkat
  - Büyüktür/küçüktür gibi basit bir koşulun yanlış seçilmesi programın bir ya da birkaç girdide yanlış çalışmasına yol açar. Uç değerler (0, 1, 10 gibi) mutlaka analiz edilmelidir.
- İlk değer ataması
  - Döngüde biriken değişkenlere (sonuç, basamak) döngüden önce doğru başlangıç değeri verilmelidir. Bu atama ayrı bir işlem kutusunda gösterilmelidir.
- Girdinin sıralı verildiğini varsaymayın
  - Soruda açıkça belirtilmedikçe kullanıcıdan verileri sıralı girmesi istenemez. Sıralama da ayrıca bir maliyettir.
- Eksik akış diyagramı çizmeyin
  - START ve STOP'u olmayan, tamamlanmamış diyagramlar yalnızca derste alternatif göstermek içindir. Ödev ve sınavlarda tam diyagram çizilmelidir.

Kısa Tekrar Notları

- Üçgen: önce a=b, sonra b=c, sonra a=c. Hiçbiri değilse çeşitkenar (tekrar sorma).
- Dik üçgen: üç Pisagor kontrolü. Eşkenar üçgen dik olamaz.
- For: adım sayısı belli. While: adım sayısı belli değil.
- For kutusu: `I ← başlangıç | artış | son değer`. Atama oku değişkene doğrudur.
- Faktöriyel: `sonuc = 1`, for I: N→1 (ya da 2→N), `sonuc = I * sonuc`. 0! = 1 doğru çıkar.
- While: koşul sağlandığı sürece döner. Koşul baştan sağlanmazsa hiç dönmez.
- Basamak sayısı: `basamak = 1`, while `N >= 10`: `N = N DIV 10`, `basamak = basamak + 1`.
- Uç değerleri test et: 0, 1, 10, negatif sayılar.

Detaylı Açıklamalar

Ders, önceki derste verilen üçgen türü probleminin çözümüyle başlar. Bir öğrencinin önerdiği "önce üçü eşit mi diye bakarım" yaklaşımı, karar kutularında yalnızca ikili karşılaştırma yapılabildiği hatırlatılarak düzeltilir. Doğru çözümde önce a ile b karşılaştırılır. Eşitlerse b ile c karşılaştırılır: eşitse üçgen eşkenar, değilse ikizkenardır. a ile b eşit değilse b ile c, o da eşit değilse a ile c karşılaştırılır. Herhangi bir eşitlik ikizkenar sonucunu verir. Üç karşılaştırma da başarısızsa üçgen çeşitkenardır ve bunu ayrıca sormaya gerek yoktur. Algoritma çeşitkenar, eşkenar ve farklı ikizkenar durumlarını temsil eden örneklerle analiz edilir.

Dik üçgen eklendiğinde Pisagor teoremi kullanılır. Hangi kenarın hipotenüs olduğu bilinmediği için üç farklı eşitlik kontrol edilmelidir. Bu kontrolün nereye konacağı tartışılır. Dik üçgen kontrolü en başa konursa her üçgen için yapılır, eşkenar bir üçgen için bile. Önce üçgenin türü belirlenip dik üçgen kontrolü sonra yapılırsa eşkenar durumda bu kontroller atlanır. Ayrıca ikizkenar durumda hangi kenarların eşit olduğu bilindiği için tek bir Pisagor kontrolü yeterli olabilir. Bu yaklaşım akış diyagramını uzatır, ama yapılan toplam işlem sayısını artırmaz. Hoca bu örnekle daha uzun bir kodun daha verimli bir algoritma olabileceğini vurgular.

Dersin ana konusu döngülerdir. Döngüler aynı işi defalarca yapmak için kullanılır. For döngüsü adım sayısı belli olan, while döngüsü adım sayısı belli olmayan işler içindir. Burada önemli bir ayrım yapılır: adım sayısının kullanıcının girdiği değere bağlı olması onu belirsiz yapmaz. Faktöriyel hesabında iş her zaman tam olarak N kez yapılır, bu yüzden for kullanılır. Bir sayının basamak sayısını bulmada ise sayının kaç kez 10'a bölüneceği önceden bilinmez, bu yüzden while kullanılır.

For döngüsü kutusunda indisin başlangıç değeri, artış miktarı ve son değeri yazılır. Döngüye giriş, her turun başlangıcı, turun sonunda başa dönüş, döngüden çıkış ve tur içindeki işlemler oklarla gösterilir. Faktöriyel algoritmasında önce N okunur ve `sonuc` değişkenine 1 atanır. Ardından I, N'den 1'e kadar birer azalarak döner ve her turda `sonuc = I * sonuc` hesaplanır. N = 5 için yapılan adım adım analiz sonucun 120 olduğunu gösterir. N = 0 girildiğinde döngüye hiç girilmez ve sonuç başlangıç değeri olan 1 olarak kalır. Bu da 0! = 1 olduğu için doğrudur. Bu yüzden `sonuc` değişkenine döngüden önce 1 atanması kritiktir. Döngünün 1 yerine 2'de bitirilmesi gereksiz bir çarpmayı önler ve tüm geçerli girdilerde doğru sonuç verir. Aynı algoritma I'nın 2'den N'e kadar artarak döndüğü biçimde de yazılabilir. Hoca, atama okunun her zaman değişkene doğru çizilmesi gerektiğini ve ters okun puan kaybına yol açacağını belirtir.

While döngüsünde koşul altıgen bir kutuya yazılır. Koşul doğru olduğu sürece döngü gövdesi tekrar eder. Koşul ilk kontrolde yanlışsa döngüye hiç girilmez. Örnekte X = 5, Y = 7 için döngü hiç dönmez. X = 7, Y = 5 için ise bir tur dönülür ve X ile Y 6'ya eşitlendiğinde döngüden çıkılır.

Basamak sayısı probleminde sayı her turda 10'a tam bölünür ve basamak sayacı bir artırılır. Sayaç 1'den başlar ve döngü N 10'dan büyük ya da eşit olduğu sürece devam eder. Koşulun sınırı dikkatle seçilmelidir. `N > 10` yazılırsa 10 sayısı yanlış olarak tek basamaklı bulunur. Sayaç 0'dan başlatılıp koşul `N >= 1` yapılırsa 0 sayısı için sonuç 0 çıkar. Derste bu biçim `N > 1` ile karıştı: bu koşulla 1 için 0, 12 için 1 çıkar. Negatif sayılar için de farklı yaklaşımlar tartışılır: sayıyı başta pozitife çevirmek ya da koşulu `N ≠ 0` yapıp sayacı 0'dan başlatmak. İkinci yöntemde 0 için ayrıca kontrol gerekir. Hoca bu örnekle, aynı problemin farklı sınır değerleriyle çözülebileceğini, ancak her değişikliğin uç değerler üzerinde mutlaka analiz edilmesi gerektiğini gösterir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.